

LoRA Fine-Tuning Projesi: Competitive Code Reasoning

Kaynaklar

LoRA Dokümantasyonu

- **LoRA Rehberi:**
<https://huggingface.co/docs/diffusers/training/lora>
- LoRA metodolojisini anlamak için mutlaka okunmalı

Dataset

Zorunlu: Bu 2 dataset'i kullanmalısınız

- **DEEP Dataset:**
<https://huggingface.co/datasets/Naholav/CodeGen-Deep-5K>
 - Daha derin reasoning trace'leri içerir
- **DIVERSE Dataset:**
<https://huggingface.co/datasets/Naholav/CodeGen-Diverse-5K>
 - Daha çeşitli problem tipleri içerir

Dataset Yapısı: Her iki dataset de şu field'ları içerir:

- input: Problem açıklaması
- output: Reasoning trace'leri + kod (<think> tag'leri içinde düşünce süreci)
- solution: Sadece temiz kod (reasoning olmadan)
- difficulty: Problem zorluk seviyesi
- source, dataset, license: Metadata bilgileri

Önemli: Her iki dataset'in de HuggingFace üzerindeki README dosyalarını mutlaka okuyun ve data yapısını anlayın

Base Model

- **Model:** <https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B-Instruct>
- **Zorunlu:** Tüm deneylerinizde bu modeli kullanmalısınız

Proje Görevleri

Görev 1: Model Kurulumu & İlk İnceleme

1. Base model'i indirin ve kurun
2. Birkaç test sorusu ile inference yapın
3. Model'in başlangıç yeteneklerini belgeleyin

Görev 2: Dataset Analizi

1. Her iki dataset'i de indirin (DEEP ve DIVERSE)
2. README dosyalarını detaylıca okuyun
3. Data yapısını lokal olarak inceleyin
4. Şu alanlar arasındaki farkı anlayın:
 - a. output field: Reasoning trace'leri içerir (<think> tag'leri)
 - b. solution field: Sadece kodu içerir

Bu projede: Training sırasında **SADECE solution field'ını** kullanacaksınız (code-only).

Opsiyonel: Reasoning yeteneklerini geliştirmek için output field'ı ile de eğitim yapabilirsiniz (şiddetle tavsiye edilir ancak zorunlu değil).

Görev 3: Training

Aynı base model'den başlayarak **2 ayrı training** yapacaksınız:

Training 1: DEEP Dataset

- Base model'den başlayın: Qwen2.5-Coder-1.5B-Instruct
- DEEP dataset'i kullanarak eğitin
- Sadece solution field'ını kullanın

Training 2: DIVERSE Dataset

- Base model'den başlayın: Qwen2.5-Coder-1.5B-Instruct (aynı base)
- DIVERSE dataset'i kullanarak eğitin
- Sadece solution field'ını kullanın

Kritik: Her iki training de base model'den sıfırdan başlıyor (birbirlerinden değil).

Görev 4: Checkpoint Seçimi

1. Training sonunda birden fazla checkpoint elde edeceksiniz (örn: checkpoint-step-400, checkpoint-step-500, checkpoint-step-600, vb.)
2. Her checkpoint'i dataset'inizin **test split'i** ile değerlendirin
- 3. Test data'yı asla training sırasında modele göstermeyin**
4. Her training için en iyi performans gösteren checkpoint'i seçin

Görev 5: Final Değerlendirme & Sunum

İlerleyen haftalarda bir benchmark dataset yayınlanacak. Ardından 1-2 sayfalık bir sunum hazırlayacaksınız:

5.1 GitHub Repository

Şunları içermeli:

- Training script
- Test/evaluation script
- Training log'ları
 - **Format:** Her 20-40 step için train loss, her 100-120 step için validation loss
 - **Önemli:** Log'ların training sırasında kaydedildiğinden emin olun
 - Terminal'e print edilen loss değerlerini mutlaka dosyaya kaydedin

5.2 HuggingFace Model Linkleri

- DEEP dataset ile eğitilmiş model
- DIVERSE dataset ile eğitilmiş model

5.3 Kullanılan Hyperparameter'lar

Tüm parametreleri belgeleyin:

- Learning rate (lr)
- LoRA rank (r)
- LoRA alpha
- Weight decay
- Optimizer ayarları
- Context length
- Batch size
- vb.

5.4 Benchmark Sonuçları

- Benchmark dataset üzerindeki final score

Önerilen Hyperparameter'lar

ÖNEMLİ NOT: Aşağıdaki tüm değerler **sadece öneridir**, zorunluluk değildir. Denemelerinize göre istediğiniz değerleri kullanabilirsiniz. Kendi deneyimlerinize göre ayarlama yapmaktan çekinmeyin.

Learning Rate (lr)

- **Kritik:** Aşırı yüksek değerler kullanmayın
- Muhafazakar başlayın ve loss eğrilerine göre ayarlayın
- Gerekirse training'i durdurup lr'yi ayarlayabilirsiniz

LoRA Konfigürasyonu

- **Rank (r):** {16, 32, 64} arasından seçin
- **Alpha:** $\alpha = r * 2$
- **Dropout:** LoRA dropout'u aktif edin
- **Target Module'ler:** LoRA'yı her ikisine de uygulayın:
 - Attention layer'lar
 - MLP layer'lar

Optimizer

- **Tip:** AdamW with fused=True
- **Scheduler:** Cosine decay
- Weight decay kullanın
- Gradient clipping kullanın (max_grad_norm)
- Warmup ratio kullanın

Training Konfigürasyonu

- **Max Epoch:** 3 (öneri - 4 de yapabilirsiniz, sadece öneridir)
- **Early Stopping:**
 - Patience = 2 veya 3 ile aktif edin
- **Context Length:**
 - Sadece solution: 1024 token
 - Reasoning ile (<think>): 8192 token
- **Effective Batch Size: 16**
 - **Önerilen strateji:** batch_size=1 + gradient_accumulation_steps=16 = effective batch size 16
 - Batch size'ı ne kadar düşürürseniz o kadar az VRAM tüketilir
 - Büyük batch etkisi elde etmek için accumulation steps'i artırın

Not: Epoch sayısı, batch size gibi değerler sadece başlangıç önerileridir. Kendi deneyimlerinize göre ayarlayabilirsiniz.

CUDA Out of Memory (OOM) Sorunları

OOM hatası alırsanız, sırayla şu çözümleri deneyin:

1. **Flash Attention 2'yi aktif edin**
2. **Gradient Checkpointing'i aktif edin**
3. **Context Length'i düşürün:**
 - a. Sadece solution: minimum 800 token
 - b. Reasoning ile: minimum 6144 token

Son Çare (hala OOM alıyorsanız):

4. 8-bit quantization kullanın:

- a. Model'i 8-bit olarak yükleyin
- b. adamw_8bit optimizer kullanın
- c.  **Uyarı:** 8-bit training, eğitim kalitesini ciddi şekilde olumsuz etkiler

Zorunlu System Prompt

Zorunlu: Tüm training'lerde bu prompt'u kullanın (sadece solution field ile eğitim için):

You are an expert Python programmer. Please read the problem carefully before writing any Python code.

Opsiyonel - Think'li Eğitim İçin: Eğer output field'ı ile reasoning eğitimi yapacaksanız, bu prompt'u kullanmalısınız:

You are an expert programmer. Use <think> tags for reasoning before writing code.

Kontrol Listesi

Teslim etmeden önce, şunların olduğundan emin olun:

- ☐ İki eğitilmiş model (DEEP ve DIVERSE)
- ☐ GitHub repository ile:
 - ☐ Training script'leri
 - ☐ Evaluation script'leri

- [] Training log'ları
- [] Her iki model için HuggingFace model card'ları
- [] Dokümante edilmiş hyperparameter'lar
- [] Test split değerlendirme sonuçları
- [] Benchmark değerlendirme sonuçları (yayınlandığında)
- [] 1-2 sayfalık sunum

Sık Karşılaşılan Sorunlar & İpuçları

Dataset Hazırlığı

- Test split'in düzgün izole edildiğinden emin olun
- Data preprocessing'in model'in beklediği formata uyduğunu kontrol edin

Training İzleme

- Her 20-40 step'te train loss'u kaydedin
- Her 100-120 step'te validation loss'u kaydedin
- Overfitting'e karşı dikkatli olun
- Learning rate sorunlarını izleyin (loss spike'ları, plateau'lar)

Değerlendirme

- Tutarlı evaluation metrikleri kullanın
- Sadece held-out data üzerinde test edin
- Tüm preprocessing adımlarını belgeleyin

Model Yükleme

- Training detaylarını içeren model card ekleyin
- Bilinen sınırlamaları belgeleyin
- Örnek kullanım kodu sağlayın

Sorularınız mı var?

Şu konularda sorun yaşarsanız veya sorularınız varsa:

- Dataset yapısı
- Training konfigürasyonu
- Evaluation metrikleri
- Model deployment

Lütfen proje raporunuzda belgelendirin.

Eğer çözemediğiniz sorular varsa: arda.mulayim@outlook.com adresine e-posta atabilirsiniz.

Ek Notlar

GPU Gereksinimleri

- **Minimum 16GB VRAM önerilir** (rahat çalışma için)
- **12GB VRAM:** Flash Attention 2 + Gradient Checkpointing kombinasyonu ile çalışabilir
- **8GB veya altı:**
 - Flash Attention 2 + Gradient Checkpointing kombinasyonu
 - Batch size = 1
 - Context length'i düşürün
 - Son çare: 8-bit quantization kullanın

Evaluation Metrikleri

- Pass@1 (kodun ilk denemede çalışma oranı)

Model Merging

- Training sonunda LoRA adapter'ları base model ile merge edebilirsiniz
- Merge edilmiş model daha hızlı inference yapar
- **Önerilir:** Merge etmeden de adapter'ı ayrı kullanabilirsiniz - bu şekilde daha esnek olursunuz

Reproducibility

- Random seed'leri sabit tutun
- Training script'inizde tüm ayarları kaydedin

Önemli Dataset Notu

Her iki dataset'te de (DEEP ve DIVERSE) bulunan problemler **NVIDIA OpenCodeReasoning** dataset'inden alınmıştır - yalnızca problem açıklamaları (input field) bu kaynaktan gelmektedir.

Reasoning ve Solution Üretimi:

- output field (reasoning trace'ler + kod): **DeepSeek API** ile kendi geliştirdiğimiz prompt kullanılarak, <think> tag'leri içinde reasoning üretilmiştir
- solution field (temiz kod): **DeepSeek API** ile oluşturulmuştur

ÇOK ÖNEMLİ - Yanlış Çözümler Hakkında:

Eğer dataset içerisinde bir problemin çözümü yanlışsa (solution field'ı hatalıysa), **lütfen o örneği dışlamayın veya filtremeyin.**

Bu çok kritik çünkü: Yanlış çözümleri filtrelemek model performansını düşürecektir. Model, yanlış çözümlerden de öğrenerek daha robust hale gelir.

Referans: <https://arxiv.org/pdf/2504.01943>

(OpenCodeReasoning paper - Section on noisy data and model robustness)

LoRA fine-tuning denemelerinizde başarılar!